Assignment No. 2

Title

Create feature branches for the project, merge them, and resolve conflicts to maintain separate workspaces for team collaboration.

Theory

Branching enables parallel development by allowing developers to work independently without affecting the main codebase. Feature branches isolate new functionality and are merged back after testing and review. Git supports fast-forward and three-way merges. Merge conflicts occur when the same code is modified differently and must be resolved manually.

Purpose

1. To understand branching concepts
2. To create and manage feature branches
3. To merge branches
4. To resolve merge conflicts
5. To enable team collaboration

Algorithms

- Create branch
- Merge branch
- Resolve conflicts

Implementation Steps

Algorithms

Algorithm 1: Create and Switch Feature Branches Steps:

- Ensure working directory is clean: git status

- Switch to main branch: git checkout main
- Update main: git pull origin main
- Create and switch to new branch: git checkout -b [feature-name]
- Make changes and stage: git add [files]
- Commit changes: git commit -m "[message]"
- Push feature branch: git push -u origin [feature-name]

Algorithm 2: Merge Feature Branch (Without Conflicts) Steps:

- Commit all changes on feature branch
- Switch to main: git checkout main
- Update main: git pull origin main
- Merge feature: git merge [feature-name]
- Push merged changes: git push origin main
- Delete local branch: git branch -d [feature-name]
- Delete remote branch: git push origin --delete [feature-name]

Algorithm 3: Resolve Merge Conflicts

Steps:

- Attempt merge: git merge [branch-name]
- Check conflicted files: git status
- Open conflicted files and locate markers: <<<<<<<, =======,
- >>>>>>>
- Resolve by editing file and removing markers
- Stage resolved file: git add [filename]
- Complete merge: git commit
- Push changes: git push origin main

Implementation Steps

Step 1: Prepare Repository

bash

- git checkout main
- git pull origin main

Step 2: Create First Feature Branch

bash

```
git checkout -b feature/data-cleaning
```

Add data cleaning functions to analysis.py: python

```
def clean_data(data ): data = data.dropna()

data = data.drop_duplicates( ) return data
```

Commit and push:

bash

```
git add analysis.py
git commit -m "Add data cleaning functions" git push -u origin feature/data-cleaning
```

Step 3: Create Second Feature Branch

bash

```
git checkout main
git checkout -b feature/visualization
```

Create visualization.py:

python

```
import matplotlib.pyplot as plt
def
   plot_distribution(data
   , column):
   plt.hist(data[column
   ]) plt.show()
```

Commit and push:

bash

```
git add visualization.py
git commit -m "Add
visualization functions" git
push -u origin
feature/visualization
```

Step 4: Merge First Branch (Clean Merge)

bash

```
git checkout main
git merge
feature/data-
cleaning git push
origin main
```

Step 5: Create Conflicting Changes

On main branch, add to analysis.py:

python

```
def main():
 print("Starting analysis pipeline...")
```

bash

```
git add analysis.py
git commit -m "Add
main function" git
push origin main
```

On visualization branch, add to same location in analysis.py:

python

```
def run_analysis():
   print("Initializing
analysis system...") bash

git checkout
feature/visualization
git add analysis.py

git commit -m "Add run_analysis function"
```

Step 6: Encounter and Resolve Conflict

Attempt merge:

bash

```
git checkout main
git merge feature/visualization
```

Git reports conflict. Check status:

bash

```
git status
```

Open analysis.py and resolve conflicts by keeping both functions, removing markers. Then:

bash

```
git add analysis.py
git commit -m "Merge visualization feature and resolve conflicts"
```

```
git push origin main
```

Step 7: Clean Up

bash

```
git branch -d feature/data-cleaning
feature/visualization
```

```
git push origin --delete feature/data-
cleaning feature/visualization
```

Application

- Parallel ML development
- A/B testing
- Feature experimentation
- Bug fixes
- Code review

Conclusion

This assignment provided hands-on experience with Git branching and conflict resolution, reinforcing collaborative development skills.